实验 3：Jena

1. 使用 Jena Fuseki 导入 music_1000_triples.nt 数据集，执行课程视频“实战-Jena.mp4”中所演示的所有 SPARQL 查询，并给出查询结果截图。

给出 music_1000_triples.nt 数据集导入成功截图：

Files to upload

+ select files | upload all

name	size	speed	status	actions
music_1000_triples.nt	98.44kb	98.64kb/s	100.00 Triples uploaded: 1000	upload now remove

(1) 查询 1

SPARQL 语句：

```
PREFIX m:<http://kg.course/music/>
SELECT DISTINCT ?trackID
WHERE {
   ?trackID m:track_artist m:artist_001
}
```

查询结果截图：

	trackID
1	<http://kg.course/music/track_00001>
2	<http://kg.course/music/track_00025>
3	<http://kg.course/music/track_00071>
4	<http://kg.course/music/track_00077>
5	<http://kg.course/music/track_00101>
6	<http://kg.course/music/track_00109>
7	<http://kg.course/music/track_00131>
8	<http://kg.course/music/track_00145>

(2) 查询 2

```
PREFIX m:<http://kg.course/music/>
SELECT ?name
WHERE {
   ?trackID m:track_artist m:artist_001 .
   ?trackID m:track_name ?name
}
```

查询结果截图：

	name
1	track_name_00001
2	track_name_00025
3	track_name_00071
4	track_name_00077
5	track_name_00101
6	track_name_00109
7	track_name_00131
8	track_name_00145

(3) 查询

```
SELECT ?trackID ?albumID ?name
WHERE {
   ?trackID m:track_name "track_name_00001" .
   ?trackID m:track_album ?albumID .
   ?albumID m:album_name ?name
}
```

查询结果截图：

	trackID	albumID	name
1	<http://kg.course/music/track_...	<http://kg.course/music/album...	album_name_0...

(4) 查询

```
SELECT ?歌曲 id ?专辑 id ?专辑名
WHERE {
   ?歌曲 id m:track_name "track_name_00001" .
   ?歌曲 id m:track_album ?专辑 id .
   ?专辑 id m:album_name ?专辑名
}
```

查询结果截图：

	歌曲id	专辑id	专辑名
1	<http://kg.course/music/track_...	<http://kg.course/music/album...	album_name_...

(5) 查询

```
SELECT ?歌曲 id ?专辑 id(CONCAT("专辑名",":",?专辑名) AS ?专辑信息)
WHERE {
  ?歌曲 id m:track_name "track_name_00001" .
  ?歌曲 id m:track_album ?专辑 id .
  ?专辑 id m:album_name ?专辑名
}
```

查询结果截图：

	歌曲id	专辑id	专辑信息
1	<http://kg.course/music/track_00001>	<http://kg.course/music/album_0001>	专辑名:album_name_0001

(6) 查询

```
SELECT ?trackID
WHERE {
  ?albumID m:album_name "album_name_0002" .
  ?trackID m:track_album ?albumID
}
```

查询结果截图：

	trackID
1	<http://kg.course/music/track_00007>
2	<http://kg.course/music/track_00010>
3	<http://kg.course/music/track_00014>

(7) 查询

```
SELECT ?trackID
WHERE {
  ?albumID m:album_name "album_name_0002" .
  ?trackID m:track_album ?albumID
}

LIMIT 2
```

查询结果截图：

	trackID
1	<http://kg.course/music/track_00007>
2	<http://kg.course/music/track_00010>

(8) 查询

```
SELECT (COUNT(?trackID)as ?num)
WHERE {
   ?albumID m:album_name "album_name_0002" .
   ?trackID m:track_album ?albumID
}
```

查询结果截图：

	num
1	"3"^^<http://www.w3.org/2001/XMLSchema#integer>

(9) 查询

```
SELECT ?trackID ?artistID
WHERE {
   ?trackID m:track_name "track_name_00001" .
   ?trackID m:track_artist ?artistID
}
```

查询结果截图：

	trackID	artistID
1	<http://kg.course/music/track_00001>	<http://kg.course/music/artist_001>

(10) 查询

```
SELECT ?trackID ?tag_name
WHERE {
   ?trackID m:track_name "track_name_00001" .
   ?trackID m:track_tag ?tag_name
}
```

查询结果截图：

	trackID	tag_name
1	<http://kg.course/music/track_00001>	tag_name_02

(11) 查询

```
SELECT ?tag_name
WHERE {
   ?trackID m:track_artist m:artist_001 .
   ?trackID m:track_tag ?tag_name
}
```

查询结果截图：

	tag_name
1	tag_name_02
2	tag_name_01
3	tag_name_08
4	tag_name_06
5	tag_name_02
6	tag_name_02
7	tag_name_06
8	tag_name_06

(12) 查询

```
SELECT DISTINCT ?tag_name
WHERE {
   ?trackID m:track_artist m:artist_001 .
   ?trackID m:track_tag ?tag_name
}
```

查询结果截图：

	tag_name
1	tag_name_02
2	tag_name_01
3	tag_name_08
4	tag_name_06

(13) 查询

```
SELECT DISTINCT ?tag_name
WHERE {
   ?trackID m:track_artist m:artist_001 .
   ?trackID m:track_tag ?tag_name
}
ORDER BY ?tag_name
```

查询结果截图：

	tag_name
1	tag_name_01
2	tag_name_02
3	tag_name_06
4	tag_name_08

(14) 查询

```
SELECT DISTINCT ?tag_name
WHERE {
   ?trackID m:track_artist m:artist_001 .
   ?trackID m:track_tag ?tag_name
}
ORDER BY DESC(?tag_name)
```

查询结果截图：

	tag_name
1	tag_name_08
2	tag_name_06
3	tag_name_02
4	tag_name_01

(15) 查询

```
SELECT (COUNT(?trackID)AS ?num)
WHERE {{
   ?trackID m:track_tag "tag_name_01"
}
UNION{
   ?trackID m:track_tag "tag_name_02"
   }}
```

查询结果截图：

	num
1	"102"^^<http://www.w3.org/2001/XMLSchema#integer>

(16) 查询

```
        SELECT (COUNT(?trackID)AS ?num)
WHERE {
   ?trackID m:track_tag ?tag_name
   FILTER (?tag_name = "tag_name_01" ||?tag_name = "tag_name_02")

}
```

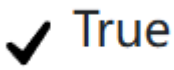
查询结果截图：

	num
1	"102"^^<http://www.w3.org/2001/XMLSchema#integer>

(17) 查询

```
        PREFIX m:<http://kg.course/music/>
ASK {
        ?trackID m:track_name ?track_name .
   FILTER regex(?track_name,"008")
}
```

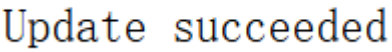
查询结果截图：



(18) 查询:

```
INSERT DATA {
  m:artist_001 m:artist_name "artist_name_001" .
  m:artist_002 m:artist_name "artist_name_002" .
  m:artist_003 m:artist_name "artist_name_003" .
}
```

查询结果截图：



（19）查询:

```
SELECT ?artistID ?artist_nmae{
?artistID m:artist_name ?artist_nmae
}
```

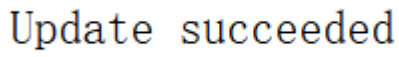
查询结果截图：

	artistID	artist_nmae
1	<http://kg.course/music/artist_001>	artist_name_001
2	<http://kg.course/music/artist_002>	artist_name_002
3	<http://kg.course/music/artist_003>	artist_name_003

（20）查询：

```
DELETE {
m:artist_002 m:artist_name ?x .
}
WHERE{
m:artist_002 m:artist_name ?x.
}
```

查询结果截图：



（21）查询：

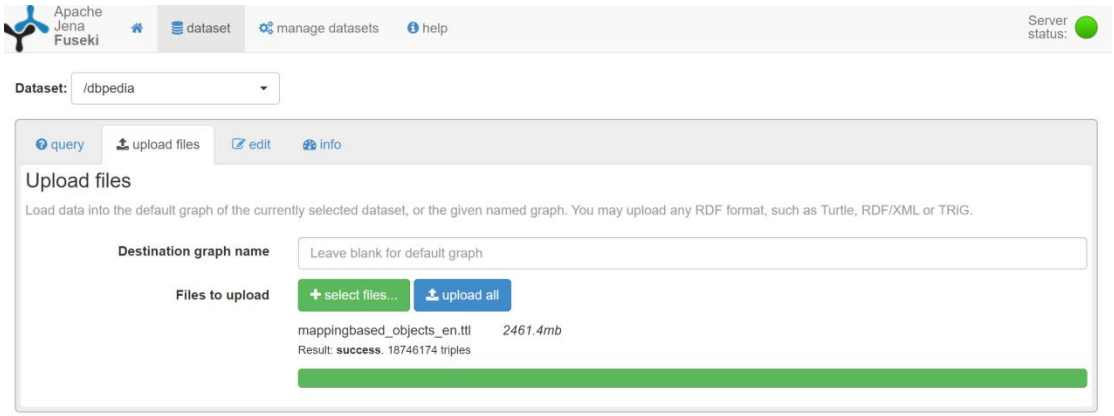
```
SELECT ?artistID ?artist_nmae{
?artistID m:artist_name ?artist_nmae
}
```

查询结果截图：

	artistID	artist_nmae
1	<http://kg.course/music/artist_001>	artist_name_001
2	<http://kg.course/music/artist_003>	artist_name_003

2. 使用 Jena Fuseki 导入 DBpedia 数据集（dbpedia-2016-10）的 mappingbased-objects_lang=en.ttl（bz2 压缩包需解压），编写 SPARQL 语句，执行以下查询问题，并给出查询结果截图。

mappingbased-objects_lang=en.ttl **数据集导入成功截图如下：**



使用以下命名空间前缀:
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX dbo: <http://dbpedia.org/ontology/>

(1) 查询 dbr:Tianjin_University 所在城市（dbo:city）以及在同一城市的类型（dbo:type）为 dbr:National_university 的其他实体（dbr:Tianjin_University 自身不包括在结果中）。

SPARQL 语句：

```
SELECT ?university ?universityName
WHERE {
   dbr:Tianjin_University dbo:city ?city .
   ?university dbo:city ?city .
   ?university dbo:type dbr:National_university .
   FILTER (?university != dbr:Tianjin_University) .

}
```

查询结果截图：

	university	universityName
1	<http://dbpedia.org/resource/Nankai_University>	
2	<http://dbpedia.org/resource/Peiyang_University>	
3	<http://dbpedia.org/resource/Tianjin_Chengjian_University>	
4	<http://dbpedia.org/resource/Tianjin_Foreign_Studies_University>	

(2) 查询 C 程序设计语言 dbr:C_\(programming_language\) 所直接和间接影响的所有其他程序设计语言（dbr:C_\(programming_language\)自身不包括在结果中）。

SPARQL 语句：SELECT DISTINCT ?influencedLanguage ?languageName
WHERE {
dbr:C_\(programming_language\) dbo:influenced* ?influencedLanguage .
FILTER (?influencedLanguage != dbr:C_\(programming_language\)) .
}

查询结果截图：

148 results in 0.015 seconds

Simple view ☐ Ellipse ☑ Filter query results Page size: 50

	influencedLanguage	languageName
1	<http://dbpedia.org/resource/C_Sharp_(programming_languag...	
2	<http://dbpedia.org/resource/Swift_(programming_language)>	
3	<http://dbpedia.org/resource/Rust_(programming_language)>	
4	<http://dbpedia.org/resource/Idris_(programming_language)>	
5	<http://dbpedia.org/resource/Crystal_(programming_language)>	
6	<http://dbpedia.org/resource/Elm_(programming_language)>	
7	<http://dbpedia.org/resource/Vue.js>	
8	<http://dbpedia.org/resource/Redux_(JavaScript_library)>	
9	<http://dbpedia.org/resource/SPARK_(programming_language)>	
10	<http://dbpedia.org/resource/Java_(programming_language)>	
11	<http://dbpedia.org/resource/PHP>	
12	<http://dbpedia.org/resource/Active_Server_Pages>	
13	<http://dbpedia.org/resource/Hack_(programming_language)>	
14	<http://dbpedia.org/resource/Jakarta_Server_Pages>	
15	<http://dbpedia.org/resource/Python_(programming_language)>	
16	<http://dbpedia.org/resource/JavaScript>	
17	<http://dbpedia.org/resource/ActionScript>	
18	<http://dbpedia.org/resource/Haxe>	
19	<http://dbpedia.org/resource/AtScript>	
20	<http://dbpedia.org/resource/TypeScript>	
21	<http://dbpedia.org/resource/CoffeeScript>	
22	<http://dbpedia.org/resource/LiveScript>	
23	<http://dbpedia.org/resource/Dart_(programming_language)>	
24	<http://dbpedia.org/resource/QML>	

共 148 条，节省起见掠过部分

(3) 查询演员 dbr:Tom_Cruise 的配偶 dbo:spouse 以及他们的出生地 dbo:birthPlace 和居住地 dbo:residence（居住地如果有则输出，如果没用就空着）。

SPARQL 语句：

```
SELECT ?spouse ?birthPlace ?residence
WHERE {
   # 1. 找到汤姆克鲁斯的所有配偶
   ?spouse dbo:spouse dbr:Tom_Cruise .

   # 2. 必查：配偶的出生地（必须有）
   ?spouse dbo:birthPlace ?birthPlace .

   # 3. 选查：配偶的居住地（没有就空着）
   OPTIONAL {
      ?spouse dbo:residence ?residence .
   }
}
```

查询结果截图：

(4) 查找爱因斯坦 dbr:Albert_Einstein 的导师（dbo:doctoralAdvisor）、导师的导师以及导师的导师的导师。

SPARQL 语句：

```
SELECT ?Advisor1   ?Advisor2 ?Advisor3
WHERE {
   dbr:Albert_Einstein dbo:doctoralAdvisor   ?Advisor1 .
   ?Advisor1 dbo:doctoralAdvisor   ?Advisor2 .
   ?Advisor2 dbo:doctoralAdvisor   ?Advisor3 .
}
```

查询结果截图：